Local Testing

```
k8s/
├── 00-namespace-config.yaml
├── 01-mongodb.yaml
├── 02-backend.yaml
└── 03-frontend.yaml
```

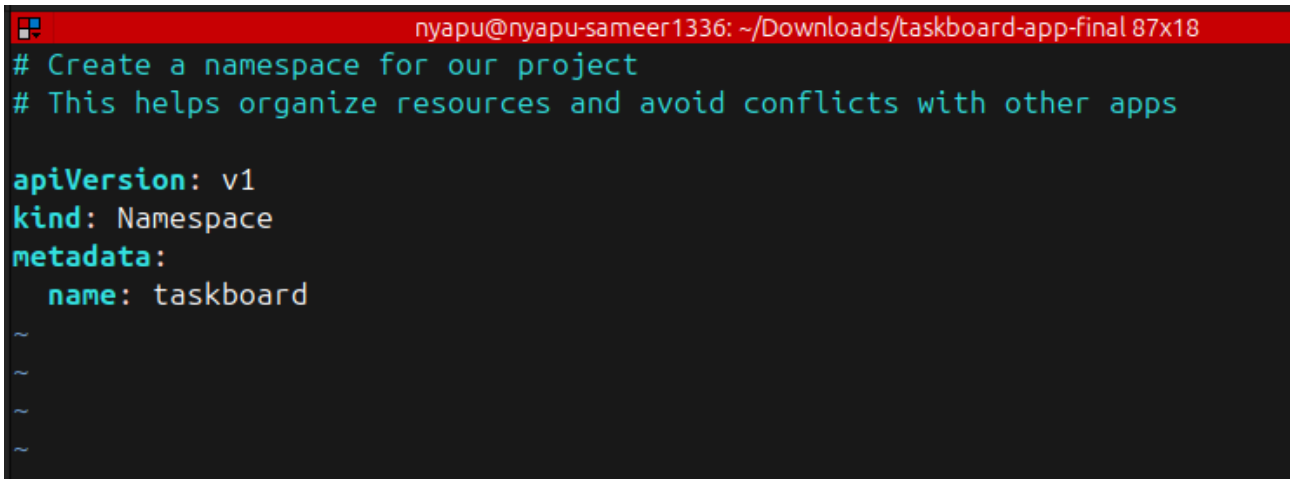
We'll make **simple Kubernetes YAML for local testing** with **clear DevOps comments** so you can teach others easily.

For image visit my profile : https://hub.docker.com/repositories/nyapu1010

```
frontend image → nyapu1010/devops-frontend
backend image  → nyapu1010/devops-backend
mongodb image  → mongo:7
```

1️⃣ Namespace (00-namespace-config.yaml)

Purpose: keep project resources isolated.



Apply first:

```
kubectl apply -f 00-namespace-config.yaml
```

2️⃣ MongoDB (01-mongodb.yaml)

MongoDB database for the app.

```yaml
# MongoDB Deployment
# Runs MongoDB container inside Kubernetes

apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongodb
  namespace: taskboard
spec:
  replicas: 1   # only one DB instance for local testing

  selector:
    matchLabels:
      app: mongodb

  template:
    metadata:
      labels:
        app: mongodb

    spec:
      containers:
        - name: mongodb

          # Official MongoDB image from DockerHub
          image: mongo:7

          ports:
            - containerPort: 27017

---

# MongoDB Service
# Allows backend to connect using DNS name "mongodb"

apiVersion: v1
kind: Service
metadata:
  name: mongodb
  namespace: taskboard

spec:
  selector:
    app: mongodb

  ports:
    - port: 27017
      targetPort: 27017

  type: ClusterIP   # internal communication only
~
~
```

Backend will connect using:

`mongodb://mongodb:27017/taskdb`

3️⃣ Backend (02-backend.yaml)

Backend API service.

```yaml
# Backend Deployment
# Runs our Node.js API container

apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: taskboard

spec:
  replicas: 1

  selector:
    matchLabels:
      app: backend

  template:
    metadata:
      labels:
        app: backend

    spec:
      containers:
        - name: backend

          # Image pulled from DockerHub
          image: nyapu1010/devops-backend

          ports:
            - containerPort: 5000

          # Environment variable used by backend
          env:
            - name: MONGO_URI
              value: mongodb://mongodb:27017/taskdb
```

.

```yaml
---

# Backend Service
# Allows frontend to talk to backend

apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: taskboard

spec:
  selector:
    app: backend

  ports:
    - port: 5000
      targetPort: 5000

  type: ClusterIP
```

Frontend will call backend using:

`http://backend:5000/api`

4 Frontend (03-frontend.yaml)

React frontend.

```yaml
# Frontend Deployment
# Runs React app inside Nginx container

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: taskboard

spec:
  replicas: 1

  selector:
    matchLabels:
      app: frontend

  template:
    metadata:
      labels:
        app: frontend

    spec:
      containers:
        - name: frontend

          # Image from DockerHub
          image: nyapu1010/devops-frontend

          ports:
            - containerPort: 80
```

.

```yaml
---

# Frontend Service
# Exposes frontend outside cluster for browser access

apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: taskboard

spec:
  selector:
    app: frontend

  ports:
    - port: 80
      targetPort: 80

  # NodePort used for local testing
  type: NodePort
```

5️⃣ Deploy Everything

From `k8s` folder:

```
kubectl apply -f .
```

Check resources:

```
kubectl get all -n taskboard
```

```
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ kubectl apply -f .
namespace/taskboard unchanged
deployment.apps/mongodb created
service/mongodb created
deployment.apps/backend created
service/backend created
deployment.apps/frontend created
service/frontend created
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ kubectl get services
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   10d
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ kubectl get services -n taskboard
NAME       TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
backend    ClusterIP   10.96.85.249    <none>        5000/TCP       17s
frontend   NodePort    10.96.188.138   <none>        80:32088/TCP   17s
mongodb    ClusterIP   10.96.68.188    <none>        27017/TCP      17s
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ kubectl get deploy -n taskboard
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
backend    1/1     1            1           67s
frontend   1/1     1            1           67s
mongodb    1/1     1            1           67s
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ kubectl get pods -n taskboard
NAME                        READY   STATUS    RESTARTS   AGE
backend-6bb4db6899-9whcp    1/1     Running   0          77s
frontend-f48fb5cc6-cqlkk    1/1     Running   0          77s
mongodb-8d66f9bd6-8nrsg     1/1     Running   0          77s
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$
```

.

Here what NodePort and ClusterIp is:

Must Know

③ NodePort Service (Kubernetes)

Learn with Sameer Tiruwa

purpose : Expose an internal service to the *outside world* by opening a specific port on *every* Node. A fundamental service type.

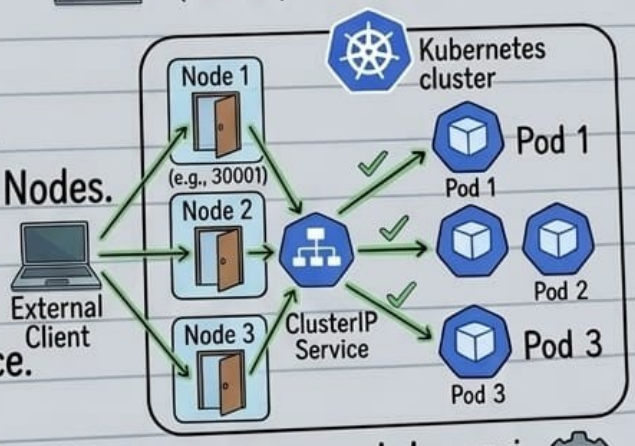
PORT

Problenges without NodePort :

- Internal IP addresses (like ClusterIP) are *not accessible* from outside the cluster.
- Clients cannot directly access Pods via their internal IPs.

Solution :

- NodePort creates a static port (30000-32767) on all Nodes.
- Traffic from the *external* client to *any* Node on this port is routed to the Service.



Benefits:

- ✓ Simple external access for testing and dev environments.
- ✓ Works with any external load balancer (creates a reliable target).
- ✓ No additional cloud provider specific infrastructure required.

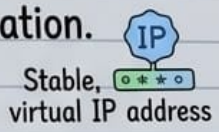
.

Must Know

② ClusterIP Service (Kubernetes)

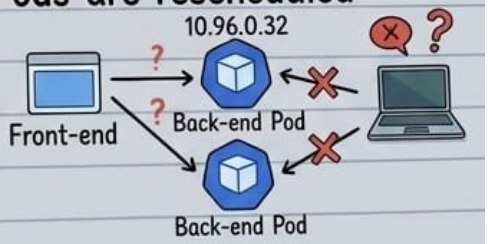
Learn with Sameer Tiruwa

purpose : Provide a single, stable IP address and DNS name for internal pod-to-pod communication. Only accessible within the cluster.



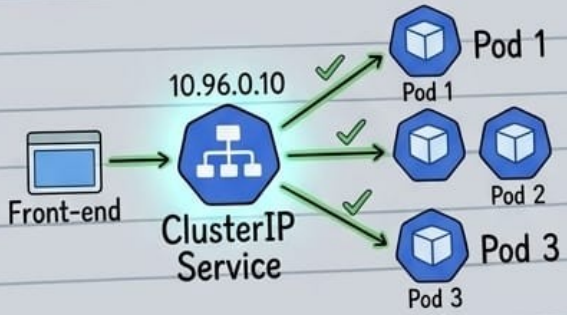
Problem before ClusterIP :

- Dynamic Pod IPs change when Pods are rescheduled (ephemeral pods).
- Front-ends can't track shifting Back-end Pod IPs.



Solution :

- ClusterIP acts as an internal load balancer and reverse proxy.
- Provides a fixed, single IP point of contact.



Benefits:

- ✓ Stable, internal service discovery.
- ✓ Automatic internal load balancing.
- ✓ Decoupled client and server pods.
- ✓ Enhanced security (no external exposure).

note: in my local tesing I have use port forwarding

6 Access the App

Find NodePort:

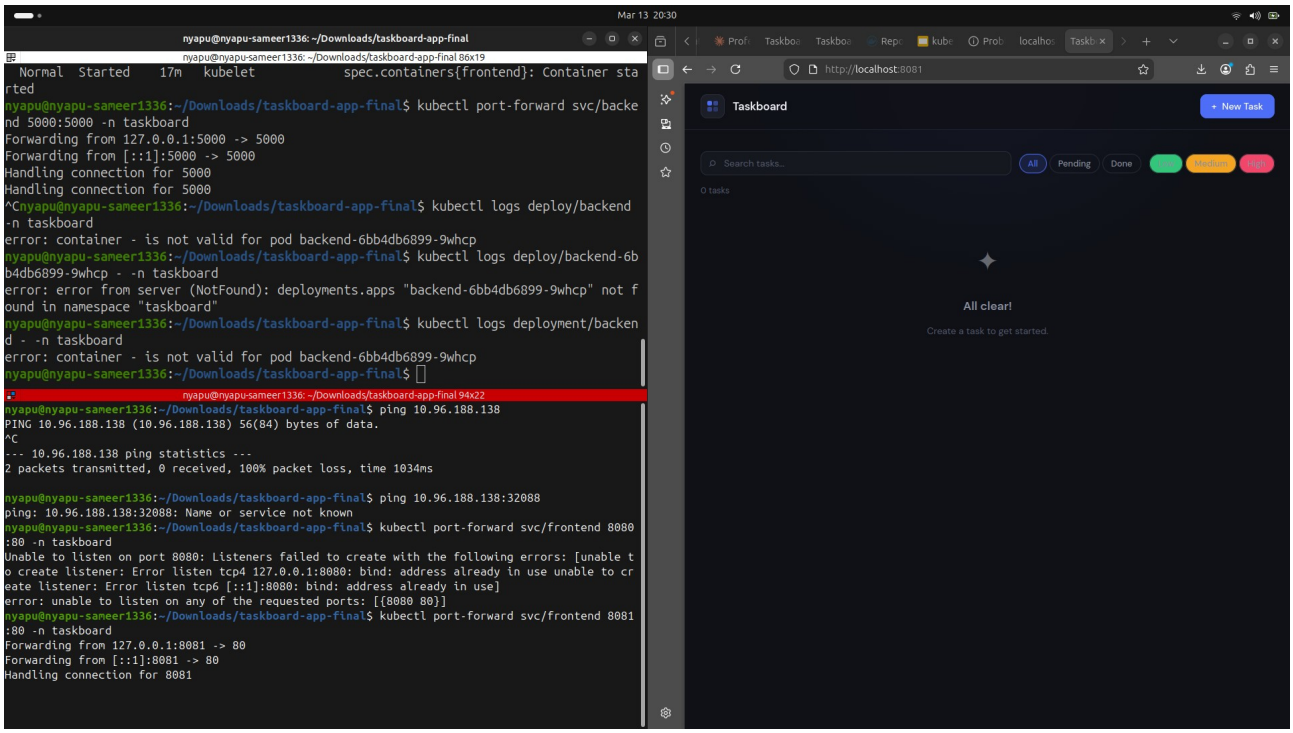
```
kubectl get svc -n taskboard
```

Example output:

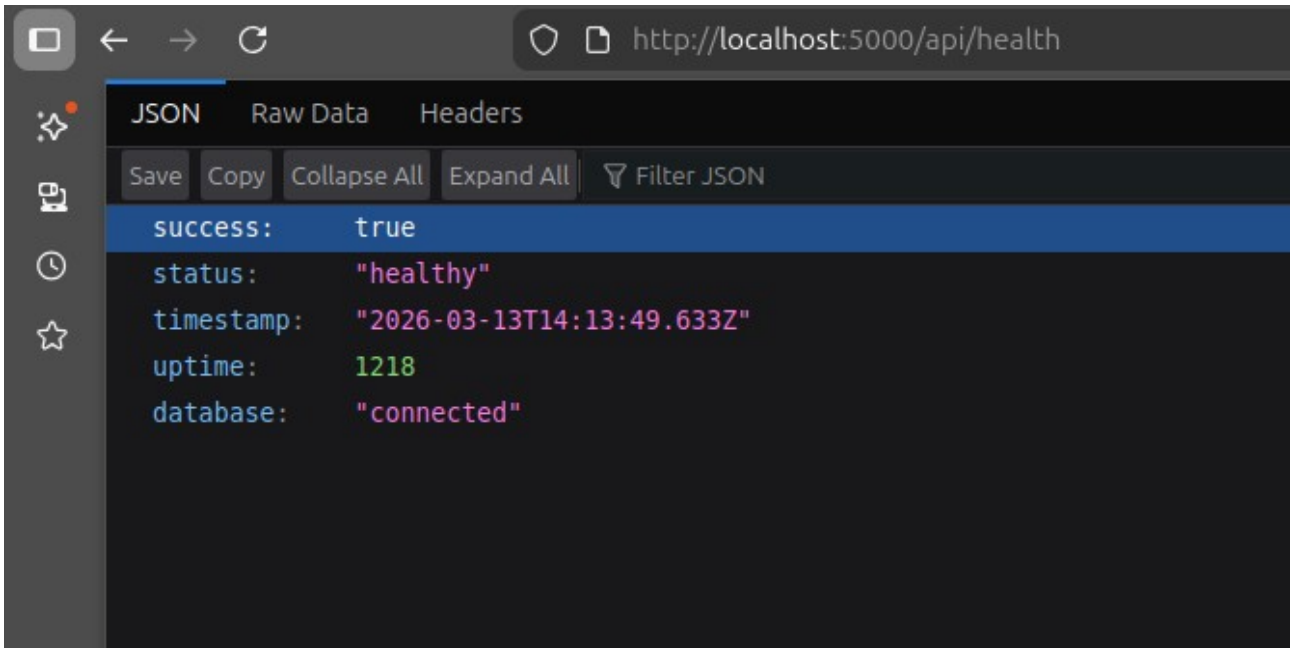
```
frontend   NodePort   80:30007
```

Open browser:

●

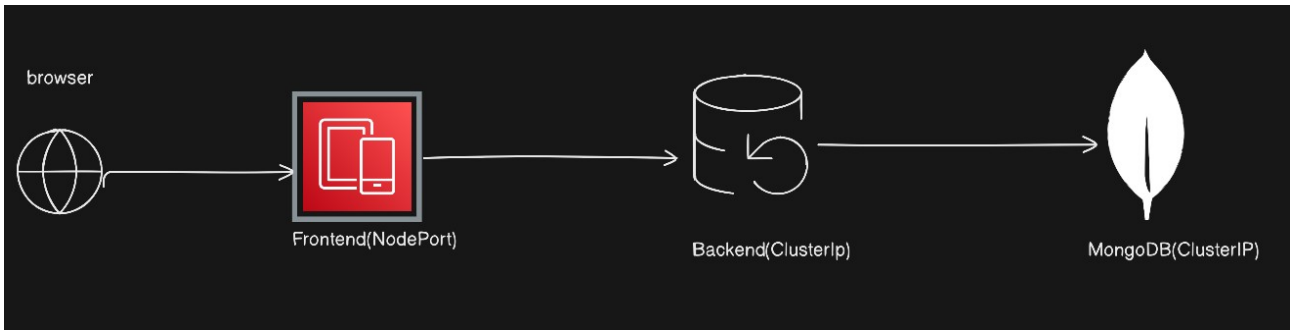

Mar 13 20:30
nyapu@nyapu-sameer1336: ~/Downloads/taskboard-app-final
nyapu@nyapu-sameer1336: ~/Downloads/taskboard-app-final 86x19
Normal Started 17m kubelet spec.containers{frontend}: Container sta
rted
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ kubectl port-forward svc/backe
nd 5000:5000 -n taskboard
Forwarding from 127.0.0.1:5000 -> 5000
Forwarding from [::1]:5000 -> 5000
Handling connection for 5000
Handling connection for 5000
^Cnyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ kubectl logs deploy/backend
-n taskboard
error: container - is not valid for pod backend-6bb4db6899-9whcp
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ kubectl logs deploy/backend-6b
b4db6899-9whcp - -n taskboard
error: error from server (NotFound): deployments.apps "backend-6bb4db6899-9whcp" not f
ound in namespace "taskboard"
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ kubectl logs deployment/backen
d - -n taskboard
error: container - is not valid for pod backend-6bb4db6899-9whcp
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$
nyapu@nyapu-sameer1336: ~/Downloads/taskboard-app-final 94x22
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ ping 10.96.188.138
PING 10.96.188.138 (10.96.188.138) 56(84) bytes of data.
^C
--- 10.96.188.138 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1034ms
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ ping 10.96.188.138:32088
ping: 10.96.188.138:32088: Name or service not known
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ kubectl port-forward svc/frontend 8080
:80 -n taskboard
Unable to listen on port 8080: Listeners failed to create with the following errors: [unable t
o create listener: Error listen tcp4 127.0.0.1:8080: bind: address already in use unable to cr
eate listener: Error listen tcp6 [::1]:8080: bind: address already in use]
error: unable to listen on any of the requested ports: [{8080 80}]
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ kubectl port-forward svc/frontend 8081
:80 -n taskboard
Forwarding from 127.0.0.1:8081 -> 80
Forwarding from [::1]:8081 -> 80
Handling connection for 8081
http://localhost:8081
Taskboard
+ New Task
Search tasks...
All
Pending
Done
Low
Medium
High
0 tasks
All clear!
Create a task to get started.

●


http://localhost:5000/api/health
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
success: true
status: "healthy"
timestamp: "2026-03-13T14:13:49.633Z"
uptime: 1218
database: "connected"

```
nyapu@nyapu-sameer1336: ~/Downloads/taskboard-app-final 173x19
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$ kubectl logs deployment/backend -n taskboard
2026-03-13 13:53:32 [info]: ✅ MongoDB connected successfully
2026-03-13 13:53:32 [info]: ✅ Server running on port 5000 [development]
2026-03-13 14:13:49 [info]: ::ffff:127.0.0.1 - - [13/Mar/2026:14:13:49 +0000] "GET /favicon.ico HTTP/1.1" 404 6
/20100101 Firefox/148.0"
nyapu@nyapu-sameer1336:~/Downloads/taskboard-app-final$
```

7⃣ Architecture (Local Kubernetes)



Follow above setup and step for **perfect local testing**.

Later we will go for **cloud production** and add:

```
Ingress Controller
Persistent Volumes
Secrets
ConfigMaps
Auto Scaling
Load Balancer(imp but we will skip for now ( cz it cost charge)
```

Made By sameer tiruwa

github:sameertiruwa1010
website: sameertiruwa.online